



PROJECT RETROSPECTIVE

PhantomGuard

How five Codex task threads turned an anti-hallucination Git-hook idea into a tested, secure, cross-platform developer tool.

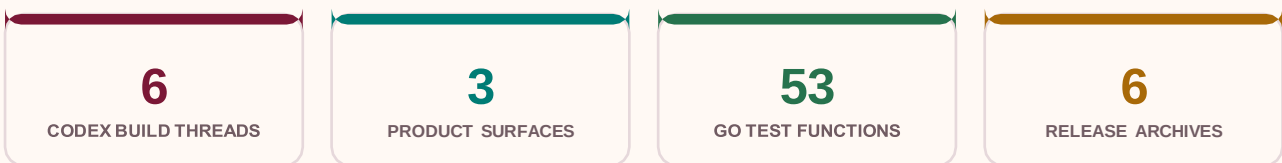
Scope note: this report intentionally excludes the current PDF-generation task. It documents the five prior PhantomGuard build threads only.

01 / Executive brief

The problem is a trust gap, not just a bad suggestion

AI coding assistants can generate plausible dependency names that do not exist. When a developer or an autonomous agent treats that text as trusted, an attacker can register the phantom identifier and turn a harmless-looking suggestion into a supply-chain entry point. This pattern is commonly discussed as **package hallucination** and, when exploited through malicious registration, **slopsquatting**.

PhantomGuard closes a precise control gap: it verifies evidence **before commit**. The tool examines the exact staged content, distinguishes definitive registry facts from uncertainty, and gives a human reviewer a visible allow, review, or block decision.



PhantomGuard is a dependency-free Go 1.21+ developer tool with a Git pre-commit hook, command-line interface, interactive terminal workspace, and local dashboard. It is designed to make the secure action the easy action.

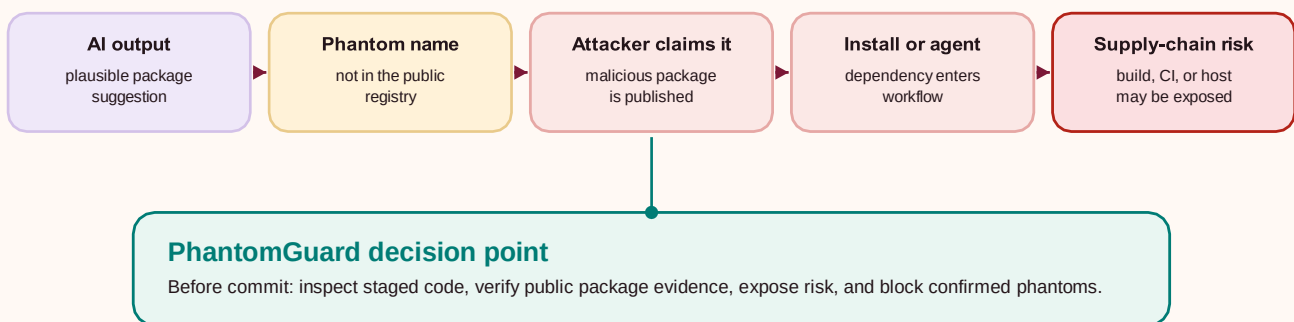
Submission thesis

AI-assisted development needs a verification gate at the moment a dependency becomes part of the software supply chain. PhantomGuard turns registry evidence, risk context, and human confirmation into that gate.

02 / Research foundation

Why package hallucinations are a cybersecurity problem

The supplied research pack places package hallucinations in the broader AI supply-chain threat model. Academic and industry sources describe how code generators can invent packages, how attackers can claim such identifiers, and why autonomous install or clone actions increase the impact. The result is not a theoretical naming error: it is an opportunity to deliver untrusted code under a name a model made plausible.



Research signals summarized from the supplied source pack:

- Code-generation research documents that package names can be hallucinated across ecosystems and models; rates vary by model, prompt, and package ecosystem. [\[1\]](#)
- MITRE ATLAS names the adversarial publication of hallucinated entities and the discovery of LLM hallucinations as relevant AI-security techniques. [\[2\]](#)
- OWASP frames supply-chain threats as a material risk for AI-enabled systems, especially when tools, plugins, skills, or dependencies are installed from third parties. [\[3\]](#)
- The risk rises when AI agents can invoke package managers, shells, repositories, or CI/CD APIs without a verification checkpoint.

Scope discipline

PhantomGuard does not claim to solve every AI-agent or software-supply-chain threat. It focuses on one controllable point: the dependency reference about to enter a Git commit.

03 / Security gap

Why traditional checks are not enough

Existing tools remain important, but they are often positioned after a dependency is resolved, installed, or known to be vulnerable. A hallucinated dependency creates a different question: *does this identifier exist, and should it be trusted at all?*

Control	Strength	Blind spot addressed by PhantomGuard
SAST	Reviews first-party source for insecure patterns.	Does not inherently prove an imported package exists in the intended public registry.
SCA	Finds known components, licenses, and vulnerabilities.	Usually starts after dependency resolution; a new malicious registration may have little historical signal.
EDR / runtime controls	Detects behavior on endpoints or workloads.	Acts later, after code is installed or executed.
Manual review	Can apply judgment and context.	Does not scale when AI tools generate dependencies at high velocity.
PhantomGuard	Checks staged references before commit.	Makes existence, uncertainty, and risk visible before the dependency lands in the repository.

Zero Trust principle:

A package name suggested by an LLM is untrusted input. A network error is also not evidence that the package exists. The tool must preserve this distinction all the way to the developer decision.

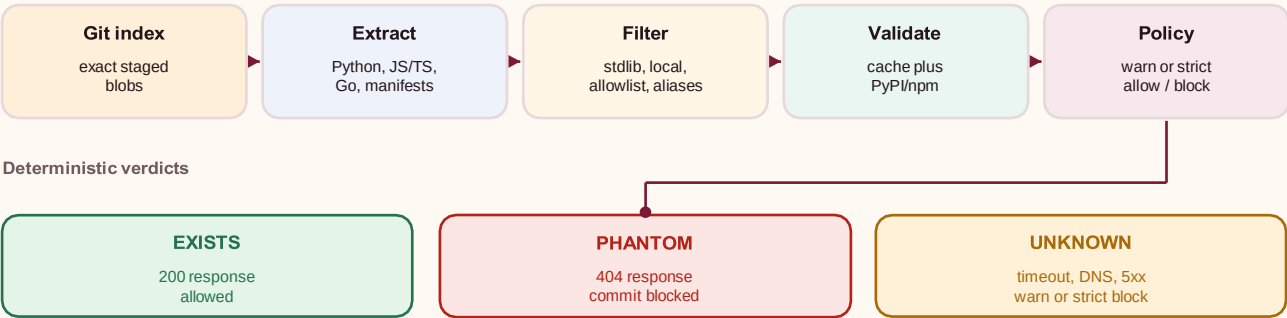
The correct failure mode

When the registry returns 404, the dependency is PHANTOM and the commit is blocked. When the network is unavailable or a response is inconclusive, the result is UNKNOWN - never silently safe.

04 / Product

PhantomGuard: evidence before dependency adoption

PhantomGuard is a static analyzer and Git pre-commit guard. It reads the Git index rather than the working tree, so the decision covers the exact content about to be committed. It never executes the scanned code.



One policy engine, three product surfaces
CLI and pre-commit hook | interactive TUI | loopback-only local dashboard

Input	What PhantomGuard extracts	What it avoids
Python	import, from import, literal importlib and __import__ calls.	Execution, dynamic evaluation, local and standard-library false positives.
JavaScript / TypeScript	Static import, export-from, require, and literal dynamic-import forms.	Node built-ins, local workspaces, aliases, comments, path and git dependencies.
Go	Native AST imports plus go.mod requirements.	False certainty: Go registry validation is intentionally UNKNOWN in v1.
Manifests	requirements files, package.json dependency sections, and go.mod.	Non-dependency file content and unsupported formats.

Make the verdict explainable

The product does not hide uncertainty behind a green check. Its policy engine returns a small set of explainable states that map directly to human action.

Verdict	Meaning	Default behavior	Strict behavior
EXISTS	The registry returned HTTP 200 for the normalized package.	Allow and cache the definitive answer.	Allow.
PHANTOM	The registry returned HTTP 404 for a public package name.	Block the commit and show exact file/line evidence.	Block.
SUSPICIOUS	The identifier is locally unsafe or malformed before any registry URL is built.	Block locally.	Block.
UNKNOWN	Timeout, DNS failure, 5xx, or another inconclusive registry state.	Allow with a visible review signal.	Block for review.

06 / Security controls

Built-in guardrails for the implementation

The implementation turns the threat model into enforceable behavior. The table below combines the project decision log, security testing, and research recommendations.

Control	Implementation decision	Why it matters
No code execution	Static parsing of staged text and manifests only.	Avoids triggering payloads while verifying a dependency reference.
Registry input validation	Reject unsafe names before constructing registry URLs.	Prevents attacker-controlled identifiers from becoming arbitrary request paths.
Bounded network use	Three-second request timeout, concurrent workers, and an eight-second overall scan budget.	Keeps the guard usable without turning a commit into an unbounded network operation.
Atomic cache	RWMutex-protected JSON cache with atomic replacement; 7-day positive and 1-hour negative TTLs.	Avoids corruption and keeps repeat scans fast without trusting uncertainty.
Typosquat signal	Damerau-Levenshtein comparison against local top-1,000 PyPI/npm datasets.	Adds context when a phantom name resembles a popular dependency.
Safe fixer	Requires clean unstaged state, verified replacement, visible diff, explicit y confirmation, and revalidation.	Keeps remediation under developer control.
AI suggestion guard	Opt-in providers only; environment credentials; independent name, registry, and risk validation.	An LLM response cannot bypass the dependency evidence gate.

AI privacy boundary

Optional AI requests are disabled by default. When enabled, the implementation sends only a small flagged source window and never stores provider keys in repository configuration.

07 / Product experience

One core policy engine, three ways to use it

The team deliberately separated interface from decision logic. CLI, TUI, and dashboard are different operating experiences over the same extraction, validation, cache, risk, and policy packages.

Surface	Use case	What it delivers
Git hook and CLI	Daily developer workflow and CI backstop.	Installable pre-commit check, exact staged scan, strict whole-project scan, cache and safe-fix commands.
Terminal workspace	Interactive developer triage without leaving the terminal.	scan staged/all/path, status, cache clear, installer, fix, version, and help with wide and compact layouts.
Local dashboard	Demonstration and investigation of findings.	Loopback-only responsive UI with live scan results, findings, policies, registry cache, and in-app walkthrough.
Release installers	Adoption without a Go toolchain.	Windows installer plus Linux/macOS installers for six packaged platform/architecture archives.

GitHub-ready commands:

Goal	Command
Install protection	<code>phantomguard install</code>
Check staged content	<code>phantomguard scan --staged</code>
Check with strict policy	<code>phantomguard scan --all --strict</code>
Open the TUI	<code>phantomguard tui</code>
Open the dashboard	<code>phantomguard serve</code>

08 / Evidence

Verification is part of the product, not an afterthought

The supplied build report and repository inspection document a verification stack that exercises both the individual security rules and the real Git workflow.

Evidence area	What was verified
Unit and package tests	Extraction, configuration, registry semantics, cache TTL/recovery/concurrency, risk scoring, AI safeguards, remediation, TUI behavior, and dashboard endpoints.
Compiled hook integration	Build binary -> install actual hook in a temporary Git repo -> stage phantom dependency -> block commit -> apply verified fix -> allow corrected commit.
Registry behavior	200 maps to EXISTS; 404 maps to PHANTOM; timeout, DNS, and 5xx map to UNKNOWN; unsafe names are rejected before HTTP.
Race and formatting checks	Race detector used where a C toolchain is available; Go vet and formatting checks are part of the quality workflow.
Docker and releases	Static platform builds, Docker image checks, installers, checksums, release packaging, and GitHub Actions verification.

Key implementation evidence:

- 13 Go test files and 53 Go test functions were observed in the project research.
- The current project test suite passed during the report-building work.
- The release toolchain packages Windows, Linux, and macOS archives for amd64 and arm64, with matching installers and checksums.
- A Docker demo path demonstrated a real PyPI 404 verdict and commit block inside an isolated container workflow.

09 / Build story

How Codex accelerated the build without replacing judgment

The Codex build report records five project threads. The pattern was iterative: define a security requirement, implement one bounded layer, run a real check, inspect the output, and revise the design when evidence demanded it.



Codex role	Concrete contribution	Human control retained
Implementation partner	Translated the brief into modular extractors, validator, cache, hook, policy, tests, docs, interfaces, and packaging.	User supplied requirements and approved the scope.
Debugging guide	Diagnosed WSL setup, Go PATH, Docker contexts, hook collisions, strict-mode configuration, and installer name conflicts.	User ran commands and shared the real output.
Safety reviewer	Preserved UNKNOWN semantics, validated the safe fixer, checked for repository secrets, and kept AI keys outside source.	User controlled provider keys, pushes, and releases.
Documentation partner	Produced README workflows, UI tutorial, demos, release guidance, and the earlier evidence-based build report.	Project team owns final claims and presentation.

Build principle

Codex did not merely generate code. It supported a repeatable loop of specification, implementation, execution evidence, correction, and documentation.

10 / Delivery

From prototype to a tool a teammate can install

The project evolved from a Python proof of concept to a supported static Go release path. The Python source remains archived for historical comparison; the active product is designed for cross-platform delivery with minimal runtime friction.

Delivery item	What is included	Submission value
Static Go binary	Dependency-free Go 1.21+ implementation under cmd/, pkg/, and data/.	Small operational footprint and predictable delivery.
Git hook installer	Writes a pre-commit hook; protects foreign hooks with a backup/chain safeguard.	Integrates at the developer decision point.
Demo scripts	PowerShell and Unix demos create an isolated repository, prove a 404, block, fix, and re-commit.	Makes the security claim demonstrable.
Dockerfile and CI	Container build, test workflows, release packaging, and installer verification.	Supports reproducibility and reviewer confidence.
Release archives	Six platform/architecture artifacts with native installers and checksum manifest.	Users can adopt without building Go.

Recommended judge demo:

- Install PhantomGuard in a temporary repository.
- Stage a deliberately unique fake dependency.
- Show the PHANTOM verdict and blocked commit.
- Show typo-risk context and the verified safe-fix preview.
- Confirm the corrected commit succeeds, then show the same result in TUI or dashboard.

11 / Limitations and roadmap

Security claims should be precise

A strong submission explains both what the tool protects and where its boundaries are. These limits are intentional design choices, not hidden failures.

Current boundary	Why it exists	Next step
Public PyPI and npm validation	The online validator is scoped to two public ecosystems for a predictable v1.	Add reviewed validators for Go and other ecosystems.
Go results can be UNKNOWN	Go imports are extracted via AST, but no Go-registry validator is used in v1.	Use strict mode now; add evidence-backed Go package validation later.
Local hooks can be bypassed	Developers can use git commit --no-verify.	Run strict scans in CI as a non-bypassable backstop.
Private registries are out of scope	Internal package namespaces need organization-specific trust configuration.	Add enterprise allowlists and private-registry integration.
Popularity corpus is finite	Top-1,000 PyPI/npm data gives useful risk context, not universal maliciousness proof.	Review and refresh datasets with provenance controls.

Roadmap direction

Expand evidence, not guesswork: lockfile and provenance checks, broader registries, CI enforcement, controlled enterprise configuration, and continued evaluation against agentic workflows.

12 / Research-to-product traceability

Every major product choice maps to a threat or control need

Research / threat observation	PhantomGuard response	Evidence in this dossier
LLMs can generate plausible nonexistent identifiers.	Static extraction and public-registry verification before commit.	Sections 2, 4, 5; reference [1].
Attackers can publish hallucinated entities and wait for selection.	404 classification, local unsafe-token rejection, and typo-risk context.	Sections 2, 5, 6; references [2], [4].
Agents can scale the impact through autonomous tools.	Commit-time decision gate, strict mode, and no-code-execution static analysis.	Sections 2, 3, 4; reference [3].
AI suggestions require independent validation.	Opt-in provider adapters plus name, registry, and risk checks before display.	Section 6 and implementation decision log.
Security controls need usable developer workflows.	Hook, CLI, TUI, dashboard, installer, Docker demo, and CI delivery paths.	Sections 7-10 and Codex build evidence.

PhantomGuard is deliberately opinionated: the dependency is not trusted because an AI suggested it, because a name looks plausible, or because a network call failed. It is trusted only when the available evidence supports that decision.

13 / References

Selected sources and project provenance

The final dossier synthesizes the three supplied PDFs. Research claims are summarized and cited below; implementation claims are traced to the supplied Codex build report and project repository materials.

Source	Use in the final dossier
[1] Spracklen et al. (2024) We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generators.	Primary research context for package hallucinations by code generators.
[2] MITRE ATLAS Publish Hallucinated Entities (AML.T0060) and related ATLAS technique material.	Threat vocabulary for adversarial publication of hallucinated resources.
[3] OWASP GenAI LLM03:2025 Supply Chain.	Supply-chain framing for AI-enabled systems.
[4] FOSSA AI Hallucinations and the New Software Supply Chain Risk.	Industry explanation of slopsquatting and practical exposure.
[5] Supplied research PDF AI Package Hallucinations, Slopsquatting, and Agentic Botnets as a Cybersecurity Threat.	11-page research pack that supplied the broader academic, standards, and industry source list.
[6] Supplied concept PDF AI Package Hallucinations and Slopsquatting concept presentation.	Original problem framing, attack-path concept, and product positioning.
[7] Supplied build report PhantomGuard Codex Build Report.	Five-thread implementation, verification, interface, and delivery evidence.

Citation note

The research pack contains a larger bibliography of 27 linked sources. This final PDF uses a focused set of representative references and retains the supplied PDFs as the detailed research record.

PhantomGuard - verify before you commit.

Final submission dossier | 21 July 2026